

Windows Toast Notification Spoofing

1. First, let's see what a Toast Notification is.

Toast Notification — used in operating systems to inform users about various application and system events. These notifications usually appear in the corner of the screen and display alerts such as a new message, an update, and similar events.

The key point here is that the notification is delivered through Windows' own notification mechanism, so users tend to perceive it as trustworthy. For example, an attacker can create a fake notification that appears to come from Microsoft Edge or Windows Defender, making it difficult for users to distinguish between legitimate and fraudulent notifications by using familiar visual elements.

Once an attacker has already gained the ability to execute code on the system, they can use the notification mechanism to manipulate the user's trust. In other words, the primary danger of this technique is not that it provides initial access to the system directly.

2. Windows Notification Architecture

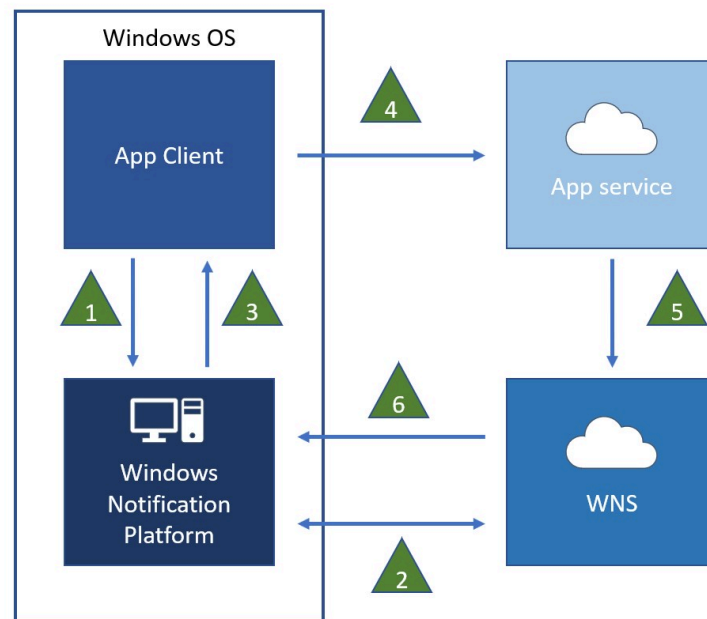
First, we need to understand how the normal Windows notification mechanism works before we can understand the spoofing technique.

A notification provides users with brief information about applications and system events. A Toast Notification is a type of brief notification window, but it is not limited to displaying text. Its structure can include a title, text, images, and interactive elements that allow users to interact with the notification. As a result, it offers a more interactive user interface element than a simple text message. If we look at the process:

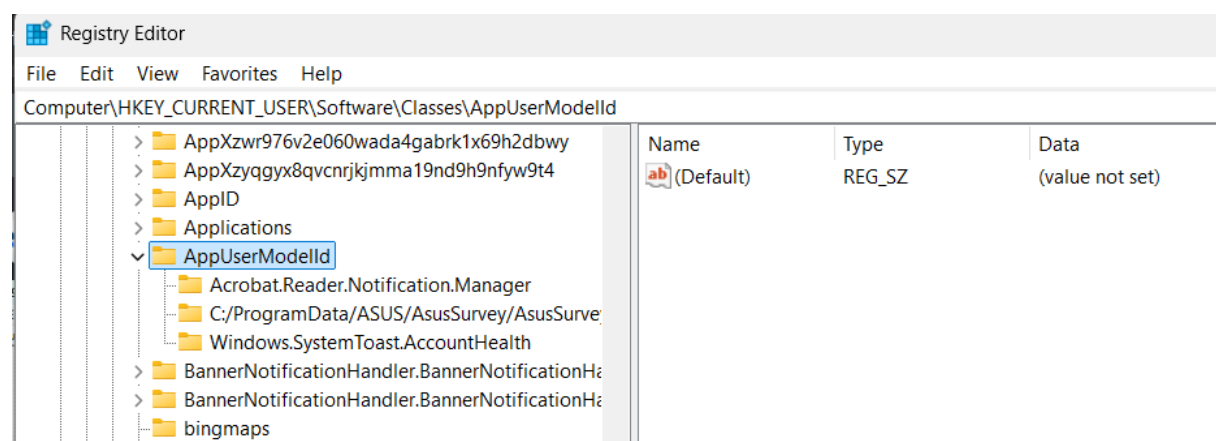
Windows Push Notification Services (WNS) is a Microsoft service that enables Windows applications to receive push notifications. WNS is primarily used in scenarios where notifications are delivered from a remote server to an

application. The application communicates with the system to generate a notification, and WNS participates in the delivery of push notifications. Windows then presents the notification to the user through the appropriate user interface.

Application → WNS → Windows → User



To understand which application a notification is associated with, it is important to understand the concept of the **Application User Model ID (AUMID)**. The application identity information stored under `HKEY_CURRENT_USER` and `HKEY_LOCAL_MACHINE` is used for this purpose.



The content presented to the user in a Toast Notification can consist of various components:

- Title and text
- Application name and application-related visual elements
- Images
- Buttons and user actions
- Input fields in certain cases
- Redirection to another resource through an action or protocol handler

These components make Toast Notifications a potential target for abuse in social engineering attacks, as attackers can exploit them to influence users into performing specific actions.

3. Threat Model

In this scenario, the attacker must already have code execution capabilities on the system (through social engineering or another method). Therefore, notification spoofing is not the initial stage of the attack chain; rather, it is considered a post-compromise technique that can be used after access has already been obtained.



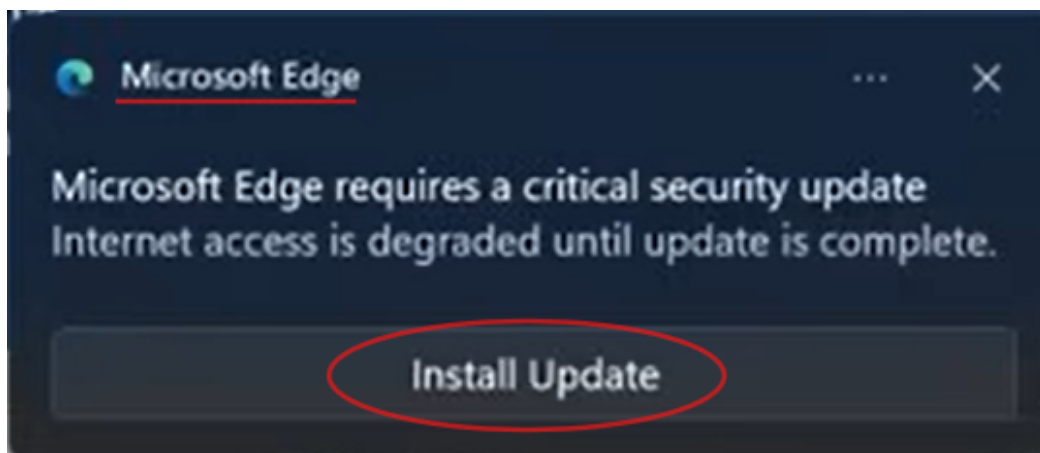
Initial Access → Code execution → Notification Abuse → Social Engineering → User Interaction → Malicious Activity

This can result in credential harvesting, redirecting the user to a phishing page, or triggering the execution of another malicious file.

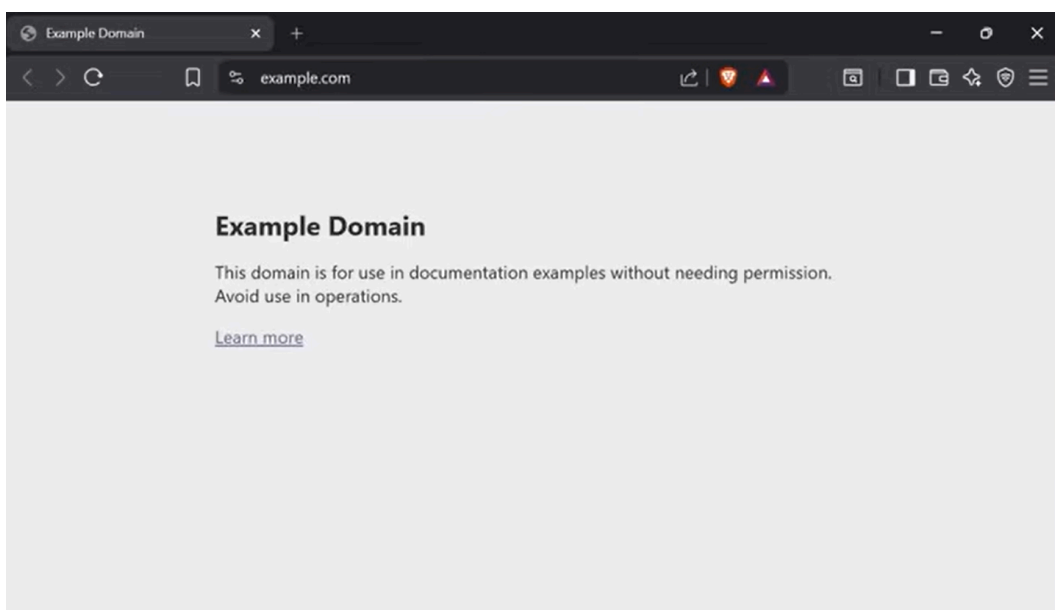
Additionally, using urgent phrases such as **"critical update," "security warning,"** or similar wording in the notification can pressure users into acting without thinking. This is a common social engineering tactic (exploits **authority, urgency, and familiarity**).

```
notifications.ps1* X
1 Add-Type -AssemblyName System.Runtime.WindowsRuntime
2
3 [Windows.UI.Notifications.ToastNotificationManager,Windows.UI.Notifications,ContentType=WindowsRunti
4 [Windows.Data.Xml.Dom.XmlDocument,Windows.Data.Xml.Dom.XmlDocument,ContentType=WindowsRuntime]
5
6 $AUMID="MSEdge"
7
8 $xml = @"
9 <toast>
10 <visual>
11 <binding template="ToastGeneric">
12 <text>Microsoft Edge requires a critical security update</text>
13 <text>Internet access is degraded until update is complete.</text>
14 </binding>
15 </visual>
16 <actions>
17 <action content="Install Update" activationType="protocol" arguments="http://example.com"/>
18 </actions>
19 </toast>
20 "@
21
22 $doc = New-Object Windows.Data.Xml.Dom.XmlDocument
23 $doc.LoadXml($xml)
```

Fake Notification Prep



Notification is shown in the expected form.

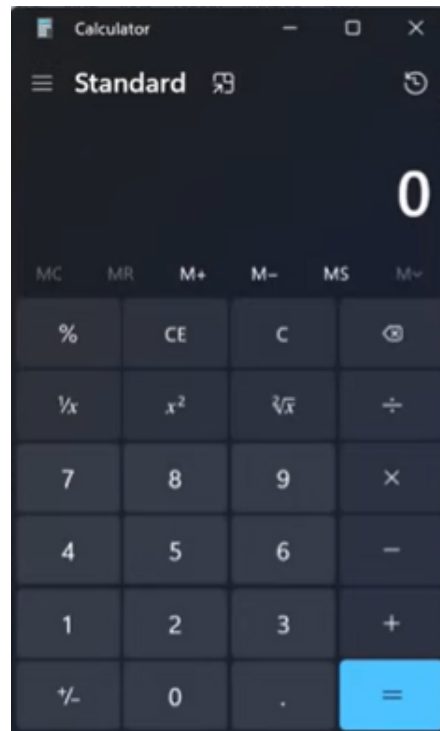


Instead of Real Update , it is forwarded to `example[.]com` .

OR

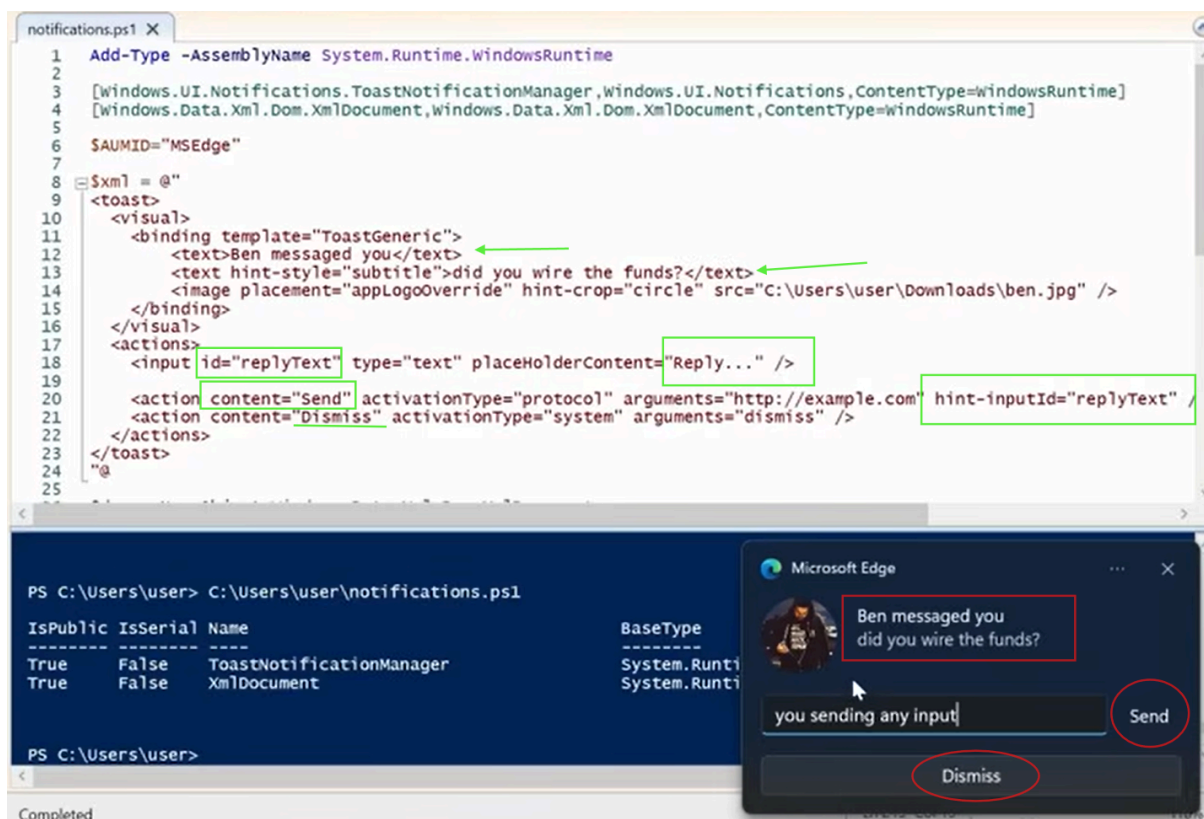
```
arguments="ms-calculator:" />
```

It is also possible to launch **Calculator** by specifying it in the argument value. The same notification will appear again, and when the user clicks it, they will be redirected to **Calculator**.



After the Click immediately

OR



This is another example.

4. Detection and Analysis

Detecting **Windows Toast Notification spoofing** differs from the traditional signature-based antivirus (AV) approach. The primary reason is that the notification mechanism itself is a legitimate part of Windows functionality. Therefore, instead of simply looking for a "malicious notification," detection should focus on identifying suspicious system behaviors that may be associated with notification activity.

Approaches such as **Endpoint Telemetry**, **Registry Monitoring**, **Process Activity**, and **Event Correlation** can all play an important role in detecting this technique.

- Audit the relevant locations in the **Windows Registry** for **AUMIDs** → Which applications are registered to send notifications? → Has a new or unexpected application identity been created? → What process activity occurred before and after the change?
- Monitor **PowerShell** activity separately.
- Collect and analyze **Sysmon** and **Windows Event Logs**.

- Review telemetry collected by **EDR agents**.
 - SIEM → Correlation Rule → ALERT
-

5. Mitigation and Prevention

If a **notification spoofing** incident is detected, the incident response process should not end with simply dismissing the notification. If malicious activity is confirmed on the endpoint, the affected endpoint and user accounts may be isolated in accordance with the organization's incident response procedures. The source of the attacker's code execution capability, as well as any associated persistence mechanisms or other malicious components, should be thoroughly investigated. The system and user accounts should be restored to a secure state, and it should be verified whether the incident has spread to other endpoints. After the incident, detection rules, logging levels, and user awareness training should be reviewed and updated as necessary.

References

- <https://www.youtube.com/watch?v=wrAFZLa1TAk>
- <https://learn.microsoft.com/en-us/windows/apps/develop/notifications/push-notifications/wns-overview>
- <https://github.com/netbiosX/ToastNotify/blob/main/README.md>
- <https://github.com/brmkit/toastnotify-bof>